

PROJECT PRODUCTION

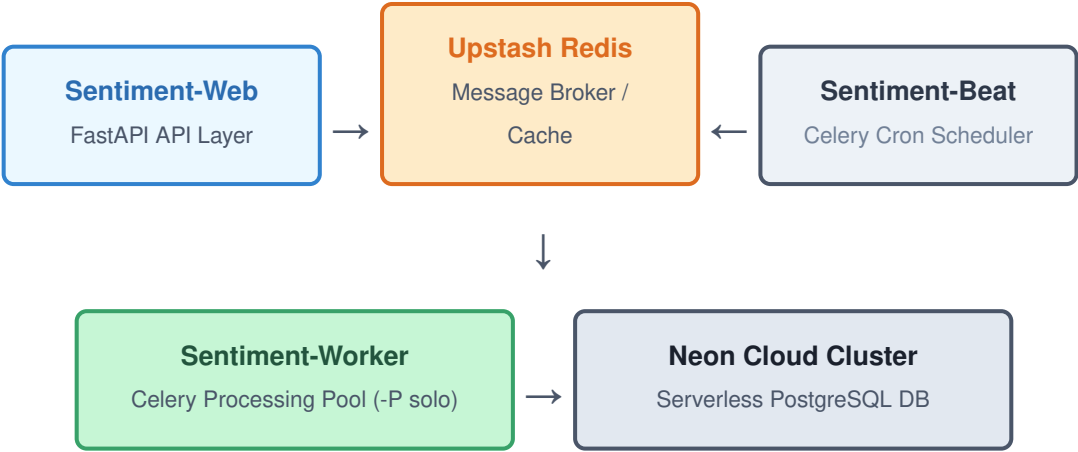
DOCUMENTATION: SENTIMENTOPS

Distributed High-Throughput Sentiment Analytics Engine & Systems Learning Ledger

System	Distributed Real-Time	Target	FastAPI, Celery, Upstash Redis, Neon
Domain:	Telemetry Pipeline	Stack:	PostgreSQL, Docker
Deployment:	Railway Cloud Infrastructure	Data Volumetrics:	19,000+ Production Datasets & Reviews

1. System Architecture Overview

The SentimentOps ecosystem is engineered as an asynchronous, non-blocking distributed processing framework capable of running heavy mathematical data computations across large datasets without interrupting primary web execution channels. By isolating compute-intensive calculations into independent worker threads, the architecture preserves low network latency overheads under peak traffic frequencies.



The operational framework is segregated into three dedicated containerized cloud runtimes:

- **Sentiment-Web (FastAPI Web Server Core):** Responsible for serving outward-facing REST API routes. It manages thin ingest routines, verifies input schemas, and instantly passes heavy multi-row computational requests downstream to the broker via a `202 Accepted` handshake to avoid UI execution stalls.
- **Sentiment-Broker & Cache (Upstash Redis Edge):** Functions as a high-speed data intermediary. It handles transient message queues for backend background processes and concurrently holds structured in-memory caches of aggregated charts to fulfill client-side analytical reads instantly.
- **Sentiment-Worker (Celery Distributed Execution Node):** A standalone computational processing box configured under single-thread isolation rules (`-P solo`). It constantly subscribes to the Redis task

registry, evaluates incoming tasks, extracts historical records out of Neon PostgreSQL, and writes the compiled telemetry matrix straight back to the memory cache.

- **Sentiment-Beat (Celery Automation Engine):** A chronological tracking process running a 180-second loop. It acts as an automated caching heart, broadcasting signals (`compute_global_stats` , `compute_distribution` , `compute_urgent_reviews`) to force background compilation loops so the frontend interface remains pre-warmed.

2. The Production Engineering Ledger (Learning Mistakes & Solutions)

Migrating a highly interactive distributed framework out of a local sandbox into a rigid cloud container matrix introduces complex configuration anomalies. Below is an engineering log documenting the system errors identified and the corresponding structural patches implemented.

BUG 1: ASYNCHRONOUS EVENT-LOOP STARTUP DEADLOCK

Symptom: The FastAPI application froze completely during deployment execution, hanging indefinitely at the lifecycle transition line `waiting for application startup` before being terminated by health checker timeouts.

Root Cause: Within the framework's startup initialization hook (`@app.on_event("startup")`), a blocking database verification query was running synchronously via `await conn.run_sync(Base.metadata.create_all)` . Under microservice constraints, this execution model locked Uvicorn's single active asynchronous loop thread, preventing the container from finishing its initialization and binding to its network socket.

Solution: Since all production data shapes had been initialized externally utilizing a dedicated migration utility (`force_neon_init.py`), verifying metadata schemas on every container launch was redundant. The blocking execution sequence was purged entirely from `app.py` , enabling immediate, millisecond-level server readiness.

BUG 2: SERVERLESS CONNECTION POOL STALLS VIA PROXY GATEWAYS

Symptom: Database query channels routinely locked up mid-session during data processing without triggering standard diagnostic database exceptions.

Root Cause: Neon routes database connections through a **PgBouncer** proxy mechanism configured in transaction pooling mode. The default setting for the asynchronous database driver `asyncpg` is to generate and optimize server-side prepared statements. In transaction pooling, sequential operations within a single session are distributed dynamically across different backend database clusters. When the driver attempted to reference a prepared statement identifier on an instance that did not generate it, the pool session deadlocked.

Solution: The connection logic inside `database.py` was adjusted to completely disable the driver's prepared statement retention cache by reducing its statement cache track metric directly to zero:

```
engine = create_async_engine(DATABASE_URL, prepared_statement_name_cache_size=0)
```

BUG 3: CROSS-OS RUNTIME CACHE CORRUPTION (WINERROR 3)

Symptom: The background execution daemon crashed continuously with system trace warnings pointing to `WinError 3: The system cannot find the path specified`.

Root Cause: During sandboxed execution on a Windows development system, Celery Beat instantiated local operational state trackers (`celerybeat-schedule.dat` , `.bak`). These binaries contained absolute Windows disk paths (`D:\Datasets\Notebooks\...`). Pushing these local cache artifacts to source control caused the Linux cloud container to read the Windows path names and throw fatal exceptions.

Solution: The tracking binaries were cleared from the remote index using `git rm --cached` , and explicit pattern exclusions were added to the `.gitignore` configuration file to shield the production environment from local operating system metadata:

```
sentimentops-backend/celerybeat-schedule*
pipeline.log
__pycache__/
```

BUG 4: CONTAINER WORKSPACE NAMESPACE COLLISIONS (MISSING MODULES)

Symptom: While the primary web endpoint container booted correctly, the worker instances collapsed instantly with a fatal `ModuleNotFoundError: No module named 'database'`.

Root Cause: The codebase is structured with all primary logic tucked cleanly into a specialized subfolder (`sentimentops-backend/`). The Dockerfile instructions established the base execution plane at `WORKDIR /app` . When Celery generated internal computation threads, it dropped its lookup scope back to the base directory path (`/app`), losing sight of sister modules like `database.py` hidden inside the subfolder.

Solution: Instead of relying on vulnerable directory change commands (`cd`) within shell scripts, the container environment parameters were appended via the cloud deployment portal to explicitly bind the backend subfolder to Linux's core structural search tracks:

```
PYTHONPATH = /app/sentimentops-backend
```

This allowed the start commands to run cleanly as direct, native calls:

```
celery -A celery_app worker --loglevel=info -P solo
```

BUG 5: BROWSER PREFLIGHT ACCESS RESTRICTIONS (CORS STALLS)

Symptom: Cloud container logs reported pristine data operations, but the interactive frontend interface remained entirely blank, outputting browser console exceptions: `Response to preflight request doesn't pass access control check` .

Root Cause: Lovable generates distinct sandbox subdomains (e.g., `*.lovableproject.com`) to handle frontend previews. When the browser dispatched an HTTP `OPTIONS` security preflight inquiry, the backend's strict CORS rules rejected the dynamic domain, blocking client-side data parsing.

Solution: Since the architecture functions as a decoupled analytical engine returning aggregate statistics, the middleware parameters were safely adjusted to allow all origins via a wildcard policy:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

3. Architectural Insights for Future Projects

Completing this complex system deployment highlights several critical principles required to engineer clean, production-grade software applications:

- **Enforce Environment Isolation Immediately:** Never permit local operational states, operating system tracking blocks, or environmental credentials to enter remote git branches. Establish a rigorous `.gitignore` blueprint at a project's inception.
- **Consolidate Base App Entrypoints:** For clean container tracking, place build scripts (`requirements.txt` , `Dockerfile`) flush with execution scripts. If structural folder nesting is mandatory, use global system path environment controls (`PYTHONPATH`) to avoid brittle directory-traversal hacks.
- **Isolate Long-Running Computation Cycles:** Retain strict non-blocking models for frontend API boundaries. Offload heavy computational aggregations onto decoupled worker infrastructure using Redis queues to maintain a snappy, high-performance user experience.

4. Final Production Status Verification

The system is fully deployed on Railway and operating within nominal performance thresholds:

- **Sentiment-Web:** `ONLINE` — Core API operational, serving live endpoints with zero event-loop blocks.
- **Sentiment-Worker:** `ONLINE` — Successfully processing asynchronous workflows and syncing task states.
- **Sentiment-Beat:** `ONLINE` — Executing automated 180-second interval cron alerts to keep data caches warmed.
- **Storage / Cache Sync:** Complete synchronization verified across all 19,000+ relational database rows on the Neon serverless cluster.